

Live demo — Member 6 — ISSD

One Ubuntu terminal; short dummy-file demonstration; genuine recorded account proof. The practical results are complete. This future classroom sequence is prepared and has not been orally rehearsed.

Every command below runs **inside ISSD-Member6-SEED12 as ordinary seed**, except the explicitly administrative setup/cleanup commands using `sudo`. See `LIVE_PREPARATION.md` for the host start action and the verified final VM state.

Part 0 — prepare before the presentation slot

0.1 Verify the existing native guest

After logging in as seed:

```
cd ~/issd-member6/lab-files
bash ../automation/check-environment.sh
sha256sum -c SOURCE_SHA256SUMS
id
file cow_attack cow_control
ls -l cow_attack cow_control
```

Expected: Ubuntu 12.04, i686/32-bit, kernel 3.5.0-37-generic, ordinary seed UID 1000, native ext4 and both ordinary seed-owned 0755 ELF32 programs. Stop on a mismatch. The existing fresh build has already been verified; if a binary is missing, rebuild using `bash build.sh` inside this guest and retain the actual output.

0.2 Start a fresh native transcript

```
LOG="../evidence/logs/demo-$(date -u +%Y%m%dT%H%M%S)-$.typescript"
test ! -e "$LOG" && script -q -f "$LOG"
```

Wait for the **new shell prompt**, then run the remaining commands in that shell. Keep its transcript path available for export. No result is inferred from starting a recorder.

```
source ../automation/guest-guard.sh
m6_require_guest && m6_require_workspace
m6_no_attacker && m6_no_process su
sha256sum -c ../provenance/passwd-normal.sha256
id charlie
```

Expected: no lab attacker/wrapper/login remains; normal file hash passes; charlie UID 1001 / GID 1002. An unexpected account change requires inspection, not a blind overwrite.

0.3 Prepare only the known lab dummy

Inspect `ls -l /zzz`. In the latest cleaned state it is absent; the prepared snapshot instead contains the identified original lab dummy. Preserve an unfamiliar file rather than overwriting it.

MANUAL ACTION REQUIRED — VM: run these administrative setup commands only after the target is absent or confirmed as the known lab dummy. Enter `sudo` passwords directly at the guest prompt and wait for each command to finish.

```
sudo sh -c 'printf "111112222233333\n" > /zzz'
sudo chown root:root /zzz
sudo chmod 0644 /zzz
sha256sum -c ../provenance/dummy-normal.sha256
```

Expected: `root:root 0644`, 19 bytes, and the original saved hash passes. This `sudo` operation is setup, not an attack result.

Create a unique trial label and preserve pre-trial metadata:

```
RUN="presentation-dummy-$(date -u +%Y%m%dT%H%M%S)-$$"
test ! -e "logs/$RUN-summary.txt"
(set -C; stat -c '%u:%g:%a:%s' /zzz > "logs/$RUN-metadata-before.txt")
```

Expected metadata is 0:0:644:19. Preserve any existing label instead of reusing it. Keep this same terminal open so \$RUN remains defined.

Part 1 — slide 4: explain and run the dummy demonstration

Clear the display only after the recording is active and preceding output has been retained. Introduce the three observations: ordinary denial, normal private copy, then the kernel race.

```
clear
id
ls -l /zzz
cat /zzz
echo 99999 > /zzz
```

Say: **“Seed is UID 1000. Root owns the file and only root has write permission. The ordinary redirection is denied.”** Expected denial is not a setup error; the file must still be original. If the write unexpectedly succeeds, stop because the protection baseline is invalid.

```
./cow_control
cat /zzz
sha256sum -c ../provenance/dummy-normal.sha256
```

Say: **“The control writes its private copy. Private memory contains stars, but the fresh backing-file read still contains the original digits.”** If the file changed or the control failed, preserve the output and investigate before the race.

```
bash run_trial.sh dummy 15 "$RUN"
cat /zzz
ls -l /zzz
```

Explain the actual outcome. The wrapper stops the attacker after detecting a change or reaching its bound. It measures integer wall seconds, not the number of repeated kernel operations. The 15-second classroom limit differs from the recorded 30-second experiment.

Complete verification, with verbose details in a separate log:

```
(set -C; stat -c '%u:%g:%a:%s' /zzz > "logs/$RUN-metadata-after.txt")
(set -C; python ../automation/verify-trial.py dummy "logs/$RUN" --live > "logs/$RUN-
verification.log" 2>&1)
echo $?
```

If exit 0: the verifier accepted the exact full result, metadata, ordinary identity and fresh file comparison. Explain why the kernel race crossed the protection boundary that the ordinary write and normal private copy respected.

If unchanged, partial, or error: keep the new summary, program, before/after and verification logs. Inspect them after the slot; say that this bounded live attempt did not establish exact success. Show **recorded S07 from 9 October**, whose sole trial measured 0 integer seconds with the 30-second bound. The saved result is identified as recorded, not attributed to the current attempt.

Part 2 — slide 5: genuine recorded account impact

Switch to the host presentation. Use S08 → S09 → S10 → S11, the matching backup slides, or video/TASK2_FALLBACK.mp4 and its clearly marked excerpts.

Say:

- **“This account experiment was recorded on 9 October 2026.”** Charlie first authenticated normally as UID 1001 / GID 1002.
- The ordinary-seed race replaced only the **four-character UID field, 1001 → 0000**. The rest of the 2040-byte file and root:root 0644 metadata stayed unchanged.
- The first authentication failed; its cause is unknown. The second **non-sudo `su - charlie`** succeeded. `id -u` returned **0** and `whoami` returned `root`. A prompt or changed hash alone would not prove this.
- Proof shell 3392 was exited before exact restoration. A new login then returned **UID 1001** and exited to seed. Restoring the file does not revoke old process credentials.

This route uses the dummy live and the completed account trial as recorded evidence. It fits the planned short segment while preserving the full Task 2 proof and restoration context.

Part 3 — clean up after every rehearsal or presentation

Wait for the wrapper to return. If interrupted, retain its output and inspect any remaining process. If a verified test login shell is open, exit it once, then verify seed; do not blindly exit an outer recorder after an authentication failure.

```
id
m6_no_attacker && m6_no_process su
sudo cmp /etc/passwd /root/issd-member6-passwd.normal
sha256sum -c ../provenance/passwd-normal.sha256
```

Only proceed with cleanup when the caller is seed, no experiment/login remains, and the account baseline matches. If a confirmed lab process remains, inspect its exact PID/command and stop that process before resetting. Do not terminate unrelated services.

MANUAL ACTION REQUIRED — VM: remove the identified dummy and run the complete checker, entering sudo authentication directly if needed.

```
sudo rm -- /zzz
bash ../automation/check-cleanup.sh
```

Expected: no attacker/wrapper/control/su/privileged shell, exact normal account/hash, only the original root:0 entry, no dummy and ordinary executables. Keep charlie normal and preserve its protected baseline for reproducibility. Capture any failure and resolve it rather than claiming cleanup.

Close the **script recording shell** with `exit` once, staying in the outer seed shell. Confirm `pgrep -x script` finds no recorder and inspect the transcript hash. Then create a fresh export:

```
bash ../automation/export-evidence.sh "demo-$(date -u +%Y%m%dT%H%M%SZ)"
```

Use Member6/VM_SESSION_GUIDE.md §11 to mount the dedicated output share, copy the completed archive/checksum, verify it on the host and unmount. Export before any snapshot rollback. Shut down the guest normally when finished. The original fresh-run evidence is already safely retained outside the VM.

Recovery and interpretation

Actual symptom	Action and explanation
Missing program/helper	Check exact VM/native path, then use the prepared guest transfer/build procedure
Pattern absent	Inspect whether the known dummy is already modified; stop processes and reset only identified lab state
Direct write succeeds	Invalid ordinary-protection baseline; fix setup before demonstrating
Timeout / no change	Preserve the finite outcome; it does not prove a fixed kernel
Partial or unexpected bytes	Stop, preserve whole copies/diff, reset the known dummy after inspection
Privileged or su process remains	Identify and end the lab shell/process before restoration or cleanup
Account baseline differs	Preserve and inspect; do not overwrite unrelated new account work
Damaged guest	Export useful evidence first; follow the explicit VirtualBox recovery in LIVE_PREPARATION

Task/source pattern: **Wenliang Du / SEED Labs, CC BY-NC-SA 4.0**. Actual assistance is disclosed in REPORT.md. A future spoken rehearsal, successful live race, or course upload is not assumed by this guide.